Lab Assignment-4

Name:	**Aalok Kumar Singh**
Roll_no:	**2401201185**
Course_Name/Code:	**Operating System Lab ENCA252**
Program_name:	**BCA (AI & DS)**
Assistant Professor:	**Dr. Sneh Lata Yadav**

Implementation and Analysis of Disk Scheduling Algorithms

Introduction

Disk scheduling is an essential function of an operating system that determines the order in which disk I/O requests are serviced. Efficient disk scheduling improves system performance by reducing seek time and increasing throughput.

In this assignment, we implemented various disk scheduling algorithms such as FCFS, SSTF, SCAN, and C-SCAN. These algorithms help in optimizing disk head movement and minimizing total seek time.

Objectives

- To understand disk scheduling concepts
- To implement various disk scheduling algorithms
- To calculate total seek time
- To compare performance of different algorithms
- To analyze system efficiency

Tools which are used

- Python 3.x
- VS Code
- Windows OS

Task 1: Input and Disk Request Representation

Description:-

In this task, we took input from the user including disk request queue, initial head position, and disk size. The request queue represents the sequence of disk I/O requests.

Code:-

```
# ---------------- INPUT ----------------
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
disk_size = int(input("Enter disk size: "))
```

Screenshot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-4-disk_scheduling>python disk_scheduling.py
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200
```

Task 2: FCFS Disk Scheduling

Description:-

FCFS (First Come First Serve) services disk requests in the order they arrive. It is simple but may result in high seek time.

Code:-

```
# ---------------- INPUT ----------------
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
disk_size = int(input("Enter disk size: "))

# ---------------- OUTPUT ----------------
print("\nResults:\n")

seek, seq = fcfs(requests, head)
print("FCFS:")
print("Sequence:", seq)
print("Seek Time:", seek)
```

```
def fcfs(requests, head):
    seek = 0
    sequence = []

    for r in requests:
        seek += abs(head - r)
        head = r
        sequence.append(r)

    return seek, sequence
```

Screenshot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-4-disk_scheduling>python disk_scheduling.py
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Results:

FCFS:
Sequence: [98, 183, 37, 122, 14, 124, 65, 67]
Seek Time: 640
```

Task 3: SSTF Algorithm

Description:-

SSTF (Shortest Seek Time First) selects the request closest to the current head position. It reduces seek time compared to FCFS.

Code:-

```python
# ---------------- INPUT ----------------
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
disk_size = int(input("Enter disk size: "))

# ---------------- OUTPUT ----------------
print("\nResults:\n")

seek, seq = sstf(requests, head)
print("\nSSTF:")
print("Sequence:", seq)
print("Seek Time:", seek)

def sstf(requests, head):
    req = requests.copy()
    seek = 0
    sequence = []

    while req:
        nearest = min(req, key=lambda x: abs(x - head))
        seek += abs(head - nearest)
        head = nearest
        sequence.append(nearest)
        req.remove(nearest)

    return seek, sequence
```

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-4-disk_scheduling>python disk_scheduling.py
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Results:


SSTF:
Sequence: [65, 67, 37, 14, 98, 122, 124, 183]
Seek Time: 236
```

Task 4: SCAN Algorithm

Description:-

SCAN (Elevator Algorithm) moves the disk head in one direction servicing all requests, then reverses direction. It provides better performance than FCFS.

Code:-

```
# ---------------- INPUT ----------------
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
disk_size = int(input("Enter disk size: "))

# ---------------- OUTPUT ----------------
print("\nResults:\n")

def scan(requests, head, disk_size):
    seek = 0
    sequence = []

    left = [r for r in requests if r < head]
    right = [r for r in requests if r >= head]

    left.sort(reverse=True)
    right.sort()

    for r in right:
        seek += abs(head - r)
        head = r
        sequence.append(r)

    # move to end
    seek += abs(head - (disk_size - 1))
    head = disk_size - 1

    for r in left:
        seek += abs(head - r)
        head = r
        sequence.append(r)

    return seek, sequence

seek, seq = scan(requests, head, disk_size)
print("\nSCAN:")
print("Sequence:", seq)
print("Seek Time:", seek)
```

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-4-disk_scheduling>python disk_scheduling.py
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Results:


SCAN:
Sequence: [65, 67, 98, 122, 124, 183, 37, 14]
Seek Time: 331
```

Task 5: C-SCAN Algorithm

Description:-

C-SCAN (Circular SCAN) moves the disk head in one direction only and jumps back to the beginning after reaching the end. It provides uniform wait time.

Code:-

```
# ---------------- INPUT ----------------
requests = list(map(int, input("Enter disk requests: ").split()))
head = int(input("Enter initial head position: "))
disk_size = int(input("Enter disk size: "))

# ---------------- OUTPUT ----------------
print("\nResults:\n")
def cscan(requests, head, disk_size):
    seek = 0
    sequence = []

    left = [r for r in requests if r < head]
    right = [r for r in requests if r >= head]

    left.sort()
    right.sort()

    for r in right:
        seek += abs(head - r)
        head = r
        sequence.append(r)

    # jump to start
    seek += abs(head - (disk_size - 1))
    head = 0
    seek += disk_size - 1

    for r in left:
        seek += abs(head - r)
        head = r
        sequence.append(r)

    return seek, sequence

seek, seq = cscan(requests, head, disk_size)
print("\nC-SCAN:")
print("Sequence:", seq)
print("Seek Time:", seek)
```

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-4-disk_scheduling>python disk_scheduling.py
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Results:


C-SCAN:
Sequence: [65, 67, 98, 122, 124, 183, 14, 37]
Seek Time: 382
```

Task 6: Performance Comparison

The performance of all algorithms was compared based on total seek time.

From the results:

- SSTF produced the least seek time
- SCAN performed better than FCFS
- C-SCAN provided uniform performance
- FCFS had the highest seek time

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-4-disk_scheduling>python disk_scheduling.py
Enter disk requests: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Results:

FCFS:
Sequence: [98, 183, 37, 122, 14, 124, 65, 67]
Seek Time: 640

SSTF:
Sequence: [65, 67, 37, 14, 98, 122, 124, 183]
Seek Time: 236

SCAN:
Sequence: [65, 67, 98, 122, 124, 183, 37, 14]
Seek Time: 331

C-SCAN:
Sequence: [65, 67, 98, 122, 124, 183, 14, 37]
Seek Time: 382
```

Result Analysis

SSTF gives the best performance in most cases as it always selects the nearest request. However, it may cause starvation.

SCAN and C-SCAN provide balanced performance and are widely used in real systems.

FCFS is simple but inefficient.

Conclusion

In this assignment, we successfully implemented disk scheduling algorithms and analyzed their performance.

The comparison helped in understanding how different algorithms affect seek time and system efficiency.